

Data Processing on Modern Hardware

Assignment 2

Handout: 7th May 2025

Due: 14th May 2025

Introduction

For this assignment we have provided a sample code-base WeeDB of a minimal variant of an in-memory database system, which is suitable for an analysis of different query execution models. The code-base contains the operators: scan, selection and aggregation for two of the processing models we covered in class: *iterator* and *materialization* models. In order to execute the queries, the query plans are first specified as a hierarchy of relational operators. In a full-fledged RDBMS, these plans would be generated with the help of the SQL parser and an optimizer.

The following query plan reads the relation R, filters the elements with values 15 and 42, and sums up the other elements.

Γ	
sum(x)	<code>//build query plan</code>
	<code>RelOperator* root;</code>
σ	<code>root = new AggregationOp (AggOp::SUM,</code>
$x \neq 15$	<code>new SelectionOp (PredType::EQUALS_NOT, 15,</code>
	<code>new SelectionOp (PredType::EQUALS_NOT, 42,</code>
σ	<code>new ScanOp (R)</code>
$x \neq 42$	<code>)</code>
	<code>);</code>
SCAN R	

The operators support the `open()`, `next()`, and `close()` interfaces for the iterator processing model and `setRelation()` interface for the materialization processing model. The following pseudo-code illustrates how to use the APIs to implement simple query plans for the two models.

<code>//execute tuple-at-a-time</code>	
<code>root->open();</code>	
<code>Tuple* t = root->next();</code>	
<code>while (t != nullptr) {</code>	<code>//execute operator-at-a-time</code>
<code> t = root->next();</code>	<code>Relation resultRelation =</code>
<code> print (t);</code>	<code>root->get();</code>
<code>}</code>	
<code>root->close();</code>	

To do the required analysis you should use a profiler. Most modern processors have so-called "performance counters", with which different cache- and CPU-events can be directly measured in hardware. These are counters with which certain events, e.g., "All requests that missed L2" can be counted. There are various tools and libraries to access the counters. For instance: Intel PCM¹ is a library that is available as open source software, works on Linux and supports very modern features of the Intel Processors.

¹<https://github.com/opcm/pcm>

Perf²³ is a tool that emerged from the Linux kernel development and also works as a standalone program. It is widely used, and has decent community support with many tutorials. For Intel CPUs you can find the list of events that can be monitored in the Intel manuals⁴.

1. Task – Analysis

Execute different queries with tuple-at-a-time and operator-at-a-time execution models. You can either use the given queries, or come up with your own. Analyze the runtime behaviour of your system and the use of the system resources like CPU, caches, main memory and memory bus to answer the following questions:

1. How heavily do the two processing models use the system's resources?
2. What changes in resource usage do you observe when you vary the number of operators? Do the differences meet your expectations? Perform some back of the envelope calculations.

The following metrics can be useful for your analysis:

- the number of memory accesses, e.g., cache lines that are moved between the main memory and the LLC (L3 cache).
- the total number of instructions that were executed.
- the number of cache hits and cache misses

2. Task – Vectorization model

The vectorization model aims to increase the efficiency of the materialization model with a better use of the CPU caches. Similar to the iterator model, the data of the relations are read and passed on from operator to operator without further I/O. However, instead of individual tuples, the operators pass blocks (i.e., vectors) of tuples to the next operator. With a suitable block size, the data remains in the CPU cache. Vector-at-a-time can thus be seen as a combination of the two processing models. This is also reflected in the interfaces of the different processing models, as shown in the figure:

```

//volcano          //vector-at-a-time          //op.-at-a-time
void open();        void openVec();              Relation getRel();
Tuple* next();      Relation* nextVec();          ←
void close();       void closeVec();

```

Extend WeeDB with the vectorization model. Implement the vector-at-a-time interface for the DBMS operators in BaseOperator.h, Operators.h, OperatorsVector.cpp and WeeDB.cpp. You can use the implementations of the other processing models as a guide.

3. Task – Analysis

Evaluate your implementation of the vector-at-a-time processing model. Expand on the previous analysis and the cost-model. Use experiments to answer the following questions:

- What influence does the block size have on the execution of vector-at-a-time?
- Can vector-at-a-time increase the cache efficiency compared to operator-at-a-time processing?

²https://perf.wiki.kernel.org/index.php/Main_Page

³<http://sandsoftwaresound.net/perf/perf-tutorial-hot-spots/>

⁴<http://www.intel.com/products/processor/manuals>

4. Deliverables

For this assignment, you are expected to:

- submit code that implements the 2nd task (files you need to modify: **BaseOperator.h**, **Operators.h**, **OperatorsVector.cpp**, **WeeDB.cpp**)
- pass the tests included in the tester file
- write a 1-page report summarizing and visualising your findings from the comparisons of the three execution models